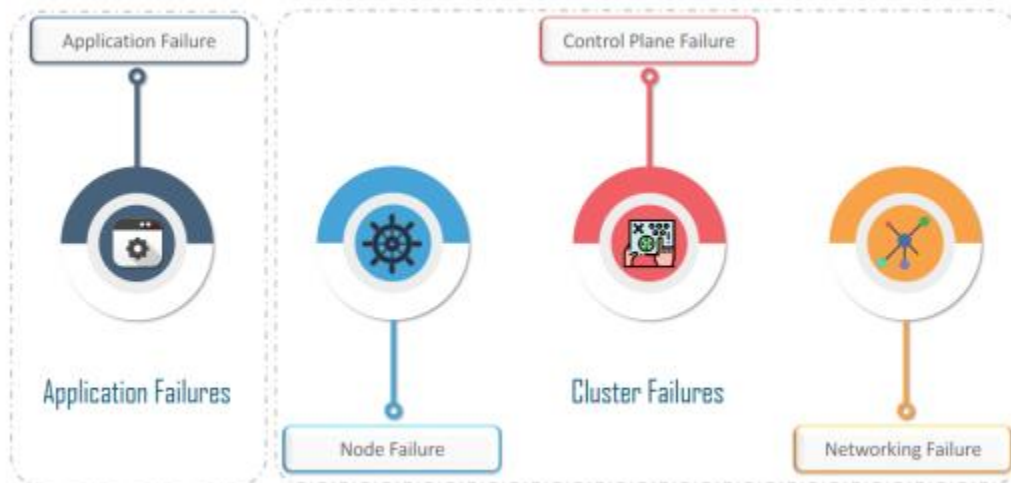


Troubleshoot:

Identify the problems is the first step in troubleshooting an application in the cluster. Problems can be identified by

- Debugging Pods
- Debugging Services
- Debugging Deployment



UC: We have 2-tier application Web and DB servers, DB server hosts on the DB pod and serves the web server through the DB-service, Web server is hosted on web pod and serves the user through web-service.

Check Service Status

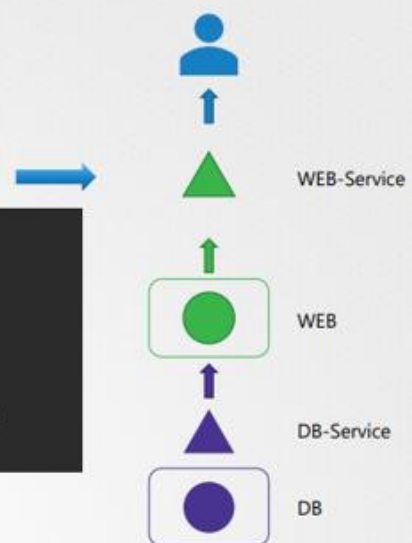
```
curl http://web-service-ip:node-port
```

```
curl: (7) Failed to connect to web-service-ip port node-port: Connection timed out
```

```
kubectl describe service web-service
```

```
Name: web-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: name=webapp-mysql
Type: NodePort
IP: 10.96.0.156
Port: <unset> 8080/TCP
TargetPort: 8080/TCP
NodePort: <unset> 31672/TCP
Endpoints: 10.32.0.6:8080
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>
```

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp-mysql
  labels:
    app: example-app
    name: webapp-mysql
spec:
  containers:
    - name: webapp-mysql
      image: simple-webapp-mysql
      ports:
        - containerPort: 8080
```



Application Failure

User reports the issue to accessing the application.

Check each and every point of this map until find the issue. In this case we start with application front end.

- Check the resources names
- Check the target-ports
- Check the endpoints
- Check the environment variables
- Check the NodePort

step1: Check the application is accessible on the IP of the NodePort?

curl <http://web-service-ip:node-port>

The diagram illustrates a user attempting to access a web service. A blue arrow points from the left towards a blue user icon. Below the icon is a green triangle labeled 'WEB-Service'. Below the triangle is a terminal window showing a failed curl command.

```
curl http://web-service-ip:node-port
```

```
curl: (7) Failed to connect to web-service-ip port node-port: Connection timed out
```

step1.1: Check the web-service

The diagram shows the output of the command 'kubectl describe service web-service' on the left, and a corresponding Kubernetes Pod configuration on the right. A blue arrow points from the service description to the pod configuration.

```
kubectl describe service web-service
```

```
Name: web-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: name=webapp-mysql
Type: NodePort
IP: 10.96.0.156
Port: <unset> 8080/TCP
TargetPort: 8080/TCP
NodePort: <unset> 31672/TCP
Endpoints: 10.32.0.6:8080
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>
```

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp-mysql
  labels:
    app: example-app
    name: webapp-mysql
spec:
  containers:
    - name: webapp-mysql
      image: simple-webapp-mysql
      ports:
        - containerPort: 8080
```

The diagram on the right shows a green triangle labeled 'WEB-Service' with a green circle labeled 'WEB' inside it. Below the circle is a blue triangle labeled 'DB-Service'.

kubectl describe svc web-service

Services: Missing Endpoints

Service might not load balance the traffic when the endpoints are not attached
Check if the endpoints are attached using the get endpoints command

Services: Labels not matched

Check the selector matches with the Pod's label.

Services: exposed Ports

If the pods listed are the expected set of Pods, there could be problem with exposed ports

Match the service's target port and the pods container port.

step1.3: Check the web Pod itself.

The screenshot shows three terminal commands and their outputs:

```
kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
Web	1/1	Running	5	50m

```
kubectl describe pod web
```

...

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	52m	default-scheduler	Successfully assigned webapp-mysql to worker-1
Normal	Pulling	52m	kubelet, worker-1	pulling image "simple-webapp-mysql"
Normal	Pulled	52m	kubelet, worker-1	Successfully pulled image "simple-webapp-mysql"
Normal	Created	52m	kubelet, worker-1	Created container
Normal	Started	52m	kubelet, worker-1	Started container

```
kubectl logs web
```

The diagram on the right shows a vertical flow of traffic: a blue circle (User) at the top, a blue arrow pointing up to a green triangle (WEB-Service), another green arrow pointing up to a green circle (WEB Pod), and a blue arrow pointing up to the WEB Pod. A blue arrow points from the terminal output to the WEB Pod.

make sure it is in **running state**, check the status as well as the number of **restarts**.

`kubectl get pods.`

check the events of the pod using describe command

`kubectl describe pod <web>`

check the logs of the application using logs command.

`kubectl logs -f web // --follow active flow`

`kubectl logs -f web -p // check the previous logs in case of restart`

step1.4: check the dependent service (db-service) and Pod (db-pod).

Control Plane Failure:

On the master node, the control plane may fail because of an issue in:

- Scheduler
- API Server
- Control Manager
- describe the component
- check the log of the component
- check the yaml of the component
- one-layer deeper docker ps
- docker logs <containerId>

step1. check the status of Control plan Pods

Check Controlplane Pods

```
kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-78fcdf6894-5dntv	1/1	Running	0	1h
coredns-78fcdf6894-knpz1	1/1	Running	0	1h
etcd-master	1/1	Running	0	1h
kube-apiserver-master	1/1	Running	0	1h
kube-controller-manager-master	1/1	Running	0	1h
kube-proxy-fvbpj	1/1	Running	0	1h
kube-proxy-v5r2t	1/1	Running	0	1h
kube-scheduler-master	1/1	Running	0	1h
weave-net-7kd52	2/2	Running	1	1h
weave-net-jt15m	2/2	Running	1	1h

step1.1 describe the status of Control plan Pods

step1.2 check the logs of Control plan Pods

Check Service Logs

```
kubectl logs kube-apiserver-master -n kube-system
```

10401 13:45:38.198735	1 server.go:703] external host was not specified, using 172.17.0.117
10401 13:45:38.194290	1 server.go:145] Version: v1.11.3
10401 13:45:38.819705	1 plugins.go:158] loaded 8 mutating admission controller(s) successfully in the following order: NamespaceLifecycle,LimitRanger,ServiceAccount,NodeRestriction,Priority,DefaultTolerationSeconds,DefaultStorageClass,MutatingAdmissionWebhook.
10401 13:45:38.819741	1 plugins.go:161] loaded 6 validating admission controller(s) successfully in the following order: LimitRanger,ServiceAccount,Priority,PersistentVolumeClaimResize,ValidatingAdmissionWebhook,ResourceQuota.
10401 13:45:38.821372	1 plugins.go:158] loaded 8 mutating admission controller(s) successfully in the following order: NamespaceLifecycle,LimitRanger,ServiceAccount,NodeRestriction,Priority,DefaultTolerationSeconds,DefaultStorageClass,MutatingAdmissionWebhook.
10401 13:45:38.821410	1 plugins.go:161] loaded 6 validating admission controller(s) successfully in the following order: LimitRanger,ServiceAccount,Priority,PersistentVolumeClaimResize,ValidatingAdmissionWebhook,ResourceQuota.
10401 13:45:38.985453	1 master.go:234] Using reconciler: lease
10401 13:45:40.900380	1 genericapiserver.go:319] Skipping API batch/v1alpha1 because it has no resources.
10401 13:45:41.370677	1 genericapiserver.go:319] Skipping API rbac.authorization.k8s.io/v1alpha1 because it has no resources.
10401 13:45:41.381736	1 genericapiserver.go:319] Skipping API scheduling.k8s.io/v1alpha1 because it has no resources.

UC: api-server is not running? – StaticPod

step1. check the status of api-server pod

step1.1 describe the status of api-server pod

step1.2 check the logs of api-server pod

UC: replicas scaling is not working?

step1. check the status of controller pod (responsible for scaling)

step1.1 describe the status of controller pod

step1.2 check the logs of controller pod

UC: Pod is in Pending state?

step1. check the status of scheduler pod (responsible for scheduling)

step1.1 describe the status of scheduler pod

step1.2 check the logs of scheduler pod

Worker node Failure:

UC: wnode is in NotReadyState?

kubectl get nodes

```
> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
worker-1	Ready	<none>	8d	v1.13.0
worker-2	NotReady	<none>	8d	v1.13.0

```
> kubectl describe node worker-1
```

```
...
Conditions:
  Type             Status      LastHeartbeatTime             Reason                       Message
  ----             -
  OutOfDisk         False       Mon, 01 Apr 2019 14:30:33 +0000 KubeletHasSufficientDisk    kubelet has sufficient disk space available
  MemoryPressure    False       Mon, 01 Apr 2019 14:30:33 +0000 KubeletHasSufficientMemory  kubelet has sufficient memory available
  DiskPressure      False       Mon, 01 Apr 2019 14:30:33 +0000 KubeletHasNoDiskPressure    kubelet has no disk pressure
  PIDPressure       False       Mon, 01 Apr 2019 14:30:33 +0000 KubeletHasSufficientPID     kubelet has sufficient PID available
  Ready             True        Mon, 01 Apr 2019 14:30:33 +0000 KubeletReady                 kubelet is posting ready status. AppArmor enabled
```

```
> kubectl describe node worker-1
```

```
...
Conditions:
  Type             Status      LastHeartbeatTime             Reason                       Message
  ----             -
  OutOfDisk         Unknown     Mon, 01 Apr 2019 14:20:20 +0000 NodeStatusUnknown           Kubelet stopped posting node status.
  MemoryPressure    Unknown     Mon, 01 Apr 2019 14:20:20 +0000 NodeStatusUnknown           Kubelet stopped posting node status.
  DiskPressure      Unknown     Mon, 01 Apr 2019 14:20:20 +0000 NodeStatusUnknown           Kubelet stopped posting node status.
  PIDPressure       False       Mon, 01 Apr 2019 14:20:20 +0000 KubeletHasSufficientPID     kubelet has sufficient PID available
  Ready             Unknown     Mon, 01 Apr 2019 14:20:20 +0000 NodeStatusUnknown           Kubelet stopped posting node status.
```

- ssh wnode01
- service docker status
- service kubelet status
- journalctl -u kubelet
- service kubelet restart
- one-layer deeper docker ps
- docker logs <containerId>

step: check the kubelet certificate, ensure they are not expired
openssl x509 -in /var/lib/kubelet/worker-1.crt -text

```
▶ openssl x509 -in /var/lib/kubelet/worker-1.crt -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      ff:e0:23:9d:fc:78:03:35
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: CN = KUBERNETES-CA
    Validity
      Not Before: Mar 20 08:09:29 2019 GMT
      Not After : Apr 19 08:09:29 2019 GMT
    Subject: CN = system:node:worker-1, O = system:nodes
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:b4:28:0c:60:71:41:06:14:46:d9:97:58:2d:fe:
        a9:c7:6d:51:cd:1c:98:b9:5e:e6:e4:02:d3:e3:71:
        58:a1:60:fe:cb:e7:9b:4b:86:04:67:b5:4f:da:d6:
        6c:08:3f:57:e9:70:59:57:48:6a:ce:e5:d4:f3:6e:
        b2:fa:8a:18:7e:21:60:35:8f:44:f7:a9:39:57:16:
        4f:4e:1e:b1:a3:77:32:c2:ef:d1:38:b4:82:20:8f:
        11:0e:79:c4:d1:9b:f6:82:c4:08:84:84:68:d5:c3:
        e2:15:a0:ce:23:3c:8d:9c:b8:dd:fc:3a:cd:42:ae:
        5e:1b:80:2d:1b:e5:5d:1b:c1:fb:be:a3:9e:82:ff:
        a1:27:c8:b6:0f:3c:cb:11:f9:1a:9b:d2:39:92:0e:
        47:45:b8:8f:98:13:c6:4d:6a:18:75:a4:01:6f:73:
```